

Práctica 1: Planificación de rutas y cobertura para un robot doméstico

En esta práctica se implementarán algoritmos de búsqueda no informada e informada para que un robot planifique sus movimientos dentro de una casa generada aleatoriamente (**CasaRobot**): llegar a un punto, hacer una ronda por varias habitaciones, o catalogar todos los objetos que detecta. Se trabajará sobre el fichero `practica1_enunciado.py`, proporcionado parcialmente implementado.

Se asume familiaridad con la API de **HouseWorld/Robot** (`common/house_world.py`): si hace falta repasarla, está documentada completa en `practica0_intro.tex` (Práctica 0). Este enunciado solo documenta lo que es *nuevo* en esta práctica.

1. Entrega

La entrega se realizará en el aula, durante la clase de prácticas del 20 de octubre de 2026, a través de la tarea habilitada para ello en el aula virtual.

El día de la entrega, una vez entregada la práctica, se cumplimentará en el aula, en modo examen y sin acceso al código, un breve cuestionario en el que se pedirá explicar cómo se ha resuelto alguna cuestión concreta de la solución entregada (ver Sección 4 para un ejemplo del estilo de pregunta que cabe esperar).

2. Infraestructura proporcionada

Todo lo de esta sección ya está implementado en `practica1_enunciado.py`: no es TODO.

2.1. La interfaz `SearchProblem`

Todos los algoritmos de esta práctica operan sobre cualquier objeto que cumpla esta interfaz —no conocen los detalles de **HouseWorld**, solo hablan con el problema a través de estos cuatro métodos—:

<code>get_start_state()</code>	devuelve el estado inicial
<code>is_goal_state(state)</code>	<code>bool</code> : si <code>state</code> es un estado objetivo
<code>get_successors(state)</code>	lista de (<code>estado_sucesor</code> , <code>accion</code> , <code>coste</code>)
<code>get_cost_of_actions(actions)</code>	coste total de ejecutar esa secuencia de acciones desde el inicio (ya implementado sobre <code>get_successors</code> , sirve para cualquier subclase)

Una acción es siempre uno de los 4 valores de `DIRECTIONS` ("N", "S", "E", "W") — igual convención que `Robot.move`, para que un plan encontrado se pueda ejecutar directamente.

2.2. `PriorityQueue`

Cola de prioridad mínima, dada: `push(item, priority)`, `pop()`, `is_empty()` ->`bool`. En caso de empate de prioridad, respeta el orden de inserción (para que el resultado no dependa del orden interno de `heapq`).

2.3. RobotPositionSearchProblem (Preguntas 1-4)

`RobotPositionSearchProblem(world, start, goal, cost_fn=None)`

Estado = posición (x,y) del robot. `cost_fn(pos)` ->float es el coste de *entrar* en pos; por defecto, coste 1 uniforme. Ya resuelto: el objetivo de las Preguntas 1 a 4 es implementar el **algoritmo** de búsqueda, no esta formulación.

2.4. room_terrain_cost(world) (para la Pregunta 3)

Devuelve una `cost_fn` ya resuelta: moverse a una celda dentro de una habitación cuesta 2 (hay que sortear muebles), moverse a una celda de pasillo cuesta 1. Se usa como `cost_fn` de un `RobotPositionSearchProblem` para la Pregunta 3 — lo que hay que implementar ahí es `uniform_cost_search`, no esta función.

2.5. manhattan_heuristic y null_heuristic (para la Pregunta 4)

`manhattan_heuristic(state, problem)` devuelve $|x - x_{goal}| + |y - y_{goal}|$; es admisible porque cada paso cuesta al menos 1. `null_heuristic(state, problem=None)` devuelve siempre 0 (con ella, A* debe comportarse exactamente como `uniform_cost_search`).

3. Desarrollo de la práctica

3.1. Paso 1: Ejecución inicial

Ejecuta el fichero tal cual se proporciona:

```
python3 practica1_enunciado.py
```

Debería generarse una casa y mostrarse con el robot colocado en ella, y las coordenadas de inicio y del objetivo de las Preguntas 1-4 (nada de lo que hay que implementar se llama todavía).

3.2. Paso 2: Pregunta 1 — depth_first_search

Implementa:

```
depth_first_search(problem) ->List[Action]
```

Búsqueda en **grafo** (no reexpande estados ya visitados) con frontera LIFO. Como distintas formas de implementarla (marcar visitado al meter en la pila o al sacar, orden de los sucesores...) dan rutas válidas pero distintas, el autograder solo comprueba que la ruta encontrada llega al objetivo sin colisionar — no que coincida con una ruta concreta.

Descomenta en el `main` el bloque del Paso 2 y observa la ruta sobre el mapa. Compárala con la que encontrarás en la Pregunta 2: ¿es tan corta como podría ser?

3.3. Paso 3: Pregunta 2 — breadth_first_search

Implementa:

```
breadth_first_search(problem) ->List[Action]
```

Búsqueda en grafo con frontera FIFO. A diferencia de la Pregunta 1, aquí sí hay un valor correcto único: el número de pasos de la ruta encontrada debe ser el mínimo posible.

3.4. Paso 4: Pregunta 3 — uniform_cost_search

Implementa:

```
uniform_cost_search(problem) ->List[Action]
```

Búsqueda en grafo con frontera de prioridad por coste acumulado desde el inicio. Pruébala con `cost_fn=room_terrain_cost(world)` (Sección 2): la ruta encontrada debe minimizar el coste total, no el número de pasos — puede (y en general debe) ser más larga en pasos que la de la Pregunta 2 si así ahorra coste evitando habitaciones.

3.5. Paso 5: Pregunta 4 — a_star_search

Implementa:

```
a_star_search(problem, heuristic=null_heuristic) ->List[Action]
```

Como `uniform_cost_search`, pero ordenando la frontera por coste acumulado **más** heurística. Pruébala con `manhattan_heuristic` sobre el mismo problema sin coste de la Pregunta 2: debe encontrar una ruta igual de corta, pero expandiendo menos nodos.

3.6. Paso 6: Preguntas 5 y 6 — ronda de inspección

Implementa la formulación del problema (Pregunta 5):

```
InspectionRouteProblem.get_start_state()
InspectionRouteProblem.is_goal_state(state)
InspectionRouteProblem.get_successors(state)
```

que representa visitar, en cualquier orden, el centro de todas las habitaciones de la casa (**checkpoints**), partiendo de **start**, minimizando el número de pasos. Estado propuesto: (**posición**, **checkpoints pendientes**) — si **start** coincide con un checkpoint, no debe contar como pendiente desde el principio.

Y la heurística (Pregunta 6):

```
inspection_route_heuristic(state, problem) ->float
```

0 si no queda ningún checkpoint pendiente; si no, la distancia Manhattan al checkpoint pendiente **más lejano** (admisble: hace falta recorrer al menos esa distancia para llegar a él, sea cual sea el orden elegido para los demás).

Prueba `InspectionRouteProblem` con `a_star_search` y tu heurística.

3.7. Paso 7: Pregunta 7 — catalogar objetos

Misma forma que el Paso 6, pero para `CatalogingProblem` (visitar todas las celdas con objetos detectados, `world.all_objects()`) y `cataloging_heuristic`. Se pide como problema independiente, con su propia formulación y heurística, porque en general habrá muchos más objetos que habitaciones y merece la pena pensar la heurística por separado.

Nota: si la casa generada no tiene ningún objeto (densidad muy baja), el `main` lo indica y no hay nada que comprobar en este paso; prueba con otra semilla si quieres verlo en acción.

4. Ejemplo ilustrativo de pregunta del cuestionario

Por ejemplo: cambiar el criterio de coste de la Pregunta 3 o la Pregunta 7 (“el robot no puede volver a pasar por una celda ya visitada salvo que sea la única forma de continuar”, o añadir una penalización por cambio de dirección) y pedir explicar, sin ver el código, si el algoritmo que has implementado seguiría siendo completo y óptimo con ese cambio, y por qué. Esto no es la pregunta real, solo un ejemplo del estilo esperado.

5. Entregables

- Un único fichero `practica1_enunciado.py` completado, renombrado a `practica1.py`, entregado en la tarea del aula virtual habilitada para ello.
- El código debe ser legible y estar debidamente tabulado. No es necesario comentarlo explicando su funcionalidad.